

is_*_type should imply is_type

Document #: P3781R0
Date: 2025-07-05
Project: Programming Language C++
Audience: Evolution Working Group
SG20 Education
Reply-to: Xavier Bonaventura ([BMW](#))

§ 1 Abstract

This paper is to raise awareness of a potential user confusion by the fact that a user needs to check for `is_type` before calling `is_*_type`. As a solution, it proposes to make the call to `is_type` implicit when calling any of the `is_*_type` functions.

§ 1.1 Tony table

Before	After
<pre>static_assert(is_class_type(^Class)); static_assert(!is_class_type(^function)); //Error: Not constexpr</pre>	<pre>static_assert(is_class_type(^Class)); static_assert(!is_class_type(^function)); //constexpr</pre>

§ 2 Motivation

§ 2.1 Motivating example

One reflection use case that is expected to be quite common is to create bindings. Different logic might be needed to create bindings depending if the entity is a class or a function. A potential naïve implementation could be as follows:

```
constexpr auto namespace_info = ^foo;  
for (int i = 0; auto element : std::meta::members_of(namespace_info,  
std::meta::access_context::current()))  
{  
    if (std::meta::is_class_type(element))  
    {  
        classes[i++] = std::meta::identifier_of(element);  
    }  
}
```

The problem with this approach is that it will not work because the call to `is_class_type` is not a constant expression.

```
if (std::meta::is_class_type(element)) // ERROR: element is not a constant  
    expression
```

It is only a constant expression if `is_type` is equal to `true`. The user would have to change the code in the following way.

```
if (std::meta::is_type(element) && std::meta::is_class_type(element))
```

But this might seem redundant.

§ 2.2 Why such confusion?

If we take the `is_class_type` function, the design decision to answer is whether it make sense this question for any entity or only for types. Let's take a real world example to answer such question.

If someone points to a lake and asks “Is it opened?” that is a nonsensical question. What does it mean a lake to be opened? Could it be closed? But if someone points to a door and asks “Is it an opened?” then that is a clear yes or no answer. However, if we change the question to “Is it an opened door?”, then it does not matter if we point to a lake or to a door, you can always answer yes or no.

Here we have a similar situation. In the moment we decided to name such functions with `is_*_type`, they semantically imply the question “Is it a type?”. Because of that, it seems confusing to have to also ask for `is_type` separately.

§ 3 Proposal

The proposal is to change the implementation of most [\[meta.reflection.traits\]](#) `is_*_type` functions to check for `is_type` when being called instead of not being `constexpr`. In case that the provided info type is not a type, they just return false.

The reason why we should change “most” of them and not all of them, is because we have some functions that accept multiple info types. For example, `is_same_type` checks if two types are the same. This implies that you provide two types. What would it mean the question if one or both are not types?

The functions in [\[meta.reflection.traits\]](#) that the name is in the form `is_*_type` that would be **included in the change** are the following ones:

```
// associated with [meta.unary.cat], primary type categories
constexpr bool is_void_type(info type);
constexpr bool is_null_pointer_type(info type);
constexpr bool is_integral_type(info type);
constexpr bool is_floating_point_type(info type);
constexpr bool is_array_type(info type);
constexpr bool is_pointer_type(info type);
constexpr bool is_lvalue_reference_type(info type);
constexpr bool is_rvalue_reference_type(info type);
constexpr bool is_member_object_pointer_type(info type);
constexpr bool is_member_function_pointer_type(info type);
constexpr bool is_enum_type(info type);
constexpr bool is_union_type(info type);
constexpr bool is_class_type(info type);
constexpr bool is_function_type(info type);
constexpr bool is_reflection_type(info type);

// associated with [meta.unary.comp], composite type categories
constexpr bool is_reference_type(info type);
constexpr bool is_arithmetic_type(info type);
```

```

constexpr bool is_fundamental_type(info type);
constexpr bool is_object_type(info type);
constexpr bool is_scalar_type(info type);
constexpr bool is_compound_type(info type);
constexpr bool is_member_pointer_type(info type);

// associated with [meta.unary.prop], type properties
constexpr bool is_const_type(info type);
constexpr bool is_volatile_type(info type);
constexpr bool is_trivially_copyable_type(info type);
constexpr bool is_trivially_relocatable_type(info type);
constexpr bool is_replaceable_type(info type);
constexpr bool is_standard_layout_type(info type);
constexpr bool is_empty_type(info type);
constexpr bool is_polymorphic_type(info type);
constexpr bool is_abstract_type(info type);
constexpr bool is_final_type(info type);
constexpr bool is_aggregate_type(info type);
constexpr bool is_consteval_only_type(info type);
constexpr bool is_signed_type(info type);
constexpr bool is_unsigned_type(info type);
constexpr bool is_bounded_array_type(info type);
constexpr bool is_unbounded_array_type(info type);
constexpr bool is_scoped_enum_type(info type);

template <reflection_range R = initializer_list<info>>
constexpr bool is_constructible_type(info type, R&& type_args);
constexpr bool is_default_constructible_type(info type);
constexpr bool is_copy_constructible_type(info type);
constexpr bool is_move_constructible_type(info type);

constexpr bool is_copy_assignable_type(info type);
constexpr bool is_move_assignable_type(info type);

constexpr bool is_swappable_type(info type);

constexpr bool is_destructible_type(info type);

template <reflection_range R = initializer_list<info>>
constexpr bool is_trivially_constructible_type(info type, R&& type_args);
constexpr bool is_trivially_default_constructible_type(info type);
constexpr bool is_trivially_copy_constructible_type(info type);
constexpr bool is_trivially_move_constructible_type(info type);

constexpr bool is_trivially_copy_assignable_type(info type);
constexpr bool is_trivially_move_assignable_type(info type);
constexpr bool is_trivially_destructible_type(info type);

template <reflection_range R = initializer_list<info>>
constexpr bool is_nothrow_constructible_type(info type, R&& type_args);
constexpr bool is_nothrow_default_constructible_type(info type);
constexpr bool is_nothrow_copy_constructible_type(info type);
constexpr bool is_nothrow_move_constructible_type(info type);

constexpr bool is_nothrow_copy_assignable_type(info type);
constexpr bool is_nothrow_move_assignable_type(info type);

constexpr bool is_nothrow_swappable_type(info type);

constexpr bool is_nothrow_destructible_type(info type);
constexpr bool is_nothrow_relocatable_type(info type);

```

```
constexpr bool is_implicit_lifetime_type(info type);

// associated with [meta.rel], type relations
template <reflection_range R = initializer_list<info>>
constexpr bool is_invocable_type(info type, R&& type_args);

template <reflection_range R = initializer_list<info>>
constexpr bool is_nothrow_invocable_type(info type, R&& type_args);
```

The functions in `[meta.reflection.traits]` that the name is in the form `is_*_type` that would be **excluded from the change** are the following ones:

```
// associated with [meta.unary.prop], type properties
constexpr bool is_assignable_type(info type_dst, info type_src);
constexpr bool is_trivially_assignable_type(info type_dst, info type_src);
constexpr bool is_nothrow_assignable_type(info type_dst, info type_src);

constexpr bool is_swappable_with_type(info type_dst, info type_src);
constexpr bool is_nothrow_swappable_with_type(info type_dst, info type_src);

// associated with [meta.rel], type relations
constexpr bool is_same_type(info type1, info type2);
constexpr bool is_base_of_type(info type_base, info type_derived);
constexpr bool is_virtual_base_of_type(info type_base, info type_derived);
constexpr bool is_convertible_type(info type_src, info type_dst);
constexpr bool is_nothrow_convertible_type(info type_src, info type_dst);
constexpr bool is_layout_compatible_type(info type1, info type2);
constexpr bool is_pointer_interconvertible_base_of_type(info type_base, info
    type_derived);

template <reflection_range R = initializer_list<info>>
constexpr bool is_invocable_r_type(info type_result, info type, R&& type_args);

template <reflection_range R = initializer_list<info>>
constexpr bool is_nothrow_invocable_r_type(info type_result, info type, R&&
    type_args);
```

§ 4 Potential polls

1. The fact that `is_*_type` functions require an additional check of `is_type` to make them constant expression might confuse user

SF	F	N	A	SA
----	---	---	---	----

2. The author should pursue the direction on making `is_type` implicit when calling `is_*_type` functions

SF	F	N	A	SA
----	---	---	---	----

§ 5 Wording

To be added in a future revision if the committee agrees that the current implementation might be confusing and the proposed solution should be pursued.